Machine Coding: Food Menu + Cart (Swiggy / Zomato)

- **Problem:** Build a restaurant menu page with a cart. Items can have **variants** (size) and **add-ons** (crust, toppings, dips). Users add, customise, increase/decrease items and see a live bill.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS) · **Time limit:** 60 minutes
- **Often asked at:** Swiggy, Zomato · **Why it matters:** item variants, add-ons, cart line identity, derived pricing

---

PART A: Thought process

The steps for any machine coding task:

1. **Understand the requirements:** what it must do, what "good" means, what is out of scope.
2. **List interactions / use cases:** every click and input with its expected result. This is your test checklist.
3. **Design the architecture:** components and who owns what.
4. **Design the state:** the minimum state; everything else is derived.
5. **Design data structures and contracts:** the shape of the API data and of the cart.
6. **Trace the data flow** for the key interaction.
7. **Find the hard parts** and solve them on paper first.
8. **Build in order:** the simplest working version first, then one working layer at a time.
9. **Cover states and edge cases:** loading, empty, error, limits.
10. **Wrap up:** trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Data model, state, component tree	8 min
Menu list + simple add / stepper (no customisation)	12 min
Customise sheet (variants + add-ons + validation)	15 min
Cart panel + bill	10 min
Repeat prompt, persistence, edge cases	7 min
Wrap-up and trade-offs	3 min

Tip: think out loud. Say *why* before you type: "I'll key cart lines by item + variant + sorted add-ons, so the same pizza with the same toppings merges into one line." The interviewer grades your reasoning as

much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show the restaurant menu grouped by **category**, each item with name, price, veg / non-veg mark, description.
- **ADD** button on an item. Once added, it becomes a **− qty +** stepper.
- Items with **variants** (Regular / Medium / Large) or **add-on groups** open a **customise sheet** before adding.
- Add-on groups have rules: *choose exactly 1* (crust) or *choose up to N* (toppings).
- The live price on the "Add item" button updates as the user picks options.
- Same item with the **same** customisation merges into one cart line; a **different** customisation is a new line.
- Pressing **+** on a customisable item asks: **Repeat last customisation** or **I'll choose**.
- Cart panel: line items with their customisation, per-line stepper, item total, delivery fee, taxes, grand total.
- Search dishes and a **Veg only** filter.
- Sold-out items cannot be added.

Non-functional requirements

Area	What we aim for
Performance	Menus are 50–300 items: plain rendering is fine. Bill and counts are derived in O(lines).
Scalability	Normalized menu (items by id) so the same item can appear in many categories and lookups are O(1).
Error handling	Loading and error-with-retry for the menu fetch; storage failures never break the cart.
Accessibility	Real radios/checkboxes inside `fieldset` + `legend`, labelled steppers, dialog with `aria-modal`, Escape to close, `aria-live` quantity.
Reusability	`QtyStepper` and `Modal` are generic; pricing logic is pure functions.
Extensibility	New add-on group = data only, no code change. Restaurant switch, coupons, checkout plug into the reducer and `getBill`.

Clarifying questions to ask

1. Does a variant **replace** the base price or **add** to it? And add-ons? *(Variant replaces, add-on adds.)*

2. When the user presses **+** on an item that has several customisations in the cart, what should happen? *(Swiggy asks "repeat last or choose new".)*
3. Is there a max quantity per item? *(Yes, 10 per line.)*
4. Can the cart hold items from more than one restaurant? *(No: ask to replace.)*
5. Should the cart survive a page refresh? *(Yes, localStorage.)*
6. Are taxes, delivery fee and free-delivery threshold fixed or from the API? *(From the API / config.)*

Assumptions we build with

- One restaurant page, menu comes from a mocked `fetchMenu()` with 500 ms delay.
- Prices are whole rupees (integers). Taxes = 5% of item total, rounded.
- Delivery fee ₹40, free above ₹499.
- Max 10 per cart line. Checkout button is a placeholder.
- Cart is persisted in localStorage.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	"Loading menu…", then categories and items. Fetch fails → message + Retry.
Click **ADD** on a simple item (Garlic Bread)	Cart line created with qty 1; button becomes stepper showing 1.
Click **+** / **−** on a simple item	Qty changes; at 0 the line is removed and ADD comes back.
Click **ADD** on a customisable item	Customise sheet opens with defaults pre-picked (first size, first crust).
Change size / crust / toppings	Price on "Add item" button updates live.
Pick a 4th topping when max is 3	Other unchecked toppings are disabled; selection doesn't change.
Required group empty	Inline error, "Add item" disabled.
Submit the sheet	Line added (or merged if same config); sheet closes.
Add the same config again (toppings clicked in another order)	Same line, qty + 1, not a duplicate line.
Add a different config	New cart line; card stepper shows the **sum** of both lines.
Click **+** on a customisable item card	"Repeat last customisation?" → Repeat (last line +1) or I'll choose (opens sheet).
Click **−** on a card with 2+ customisations	Tell the user to remove the specific one from the cart.
Cart stepper + / −	Changes that line only; + disabled at 10.
Clear cart	Empty-cart state.

Action / input	Expected behavior / output
Search "pizza" / toggle Veg only	Filters items; empty categories hidden; nothing matches → "No dishes match".
Sold-out item	Shows "Sold out", no ADD. Persisted line for it is shown greyed and excluded from the bill.
Refresh the page	Cart restored from localStorage.
Item total below ₹499	Delivery ₹40 + "Add ₹X more for free delivery". At ₹499+ → FREE.
Escape / click backdrop in a sheet	Sheet closes, nothing added.

---

3. Design the architecture

Module	Responsibility
`data/menu.js`	Mock API: normalized menu + `fetchMenu()`.
`utils/cart.js`	Pure logic: line key, unit price, add-on toggle, validation, bill, formatting.
`context/cartReducer.js`	Pure state transitions: ADD / INC / DEC / CLEAR.
`context/CartContext.jsx`	Holds cart state, persists it, exposes derived values (qty per item, bill) and actions.
`App`	Fetches menu, handles loading / error, mounts provider.
`Menu`	Search + veg filter, category sections, owns which sheet is open.
`MenuItemCard`	ADD button or stepper, decides "simple add" vs "customise" vs "repeat".
`CustomizeModal`	Local selection state, variant radios, add-on groups, live price, validation.
`RepeatPrompt`	Repeat last customisation vs choose new.
`Cart`	Lines, per-line stepper, bill breakdown.
`QtyStepper`, `Modal`	Generic UI pieces.

```
App  (menu fetch: loading | error | success)
└── CartProvider  (cart state + localStorage + derived bill)
    ├── Menu  (query, vegOnly, customizing, repeatFor)
    │   ├── MenuItemCard × N ── QtyStepper
    │   ├── CustomizeModal ── Modal   (local selection)
    │   └── RepeatPrompt   ── Modal
    └── Cart ── QtyStepper × lines
```

Start simple: first version is `App` + `Menu` + card + cart with simple items only. Split out `CustomizeModal`, `RepeatPrompt` and `Modal` only once variants/add-ons come in.

4. Design the state / data model

State or field	Owner	Purpose
`status`, `menu`	`App`	Fetch lifecycle and server data (read-only).
`cart.restaurantId`	Cart context	Which restaurant the cart belongs to.
`cart.lines[]`	Cart context	`{ key, itemId, variantId, addonIds, qty }`: the only cart truth.
`query`, `vegOnly`	`Menu`	Filters (UI-only).
`customizing`, `repeatFor`	`Menu`	Which item's sheet is open (only one at a time).
`sel = { variantId, addonIds }`	`CustomizeModal`	Draft selection; thrown away on close.

```
// cart state (reducer)
const initialCart = { restaurantId: null, lines: [] };

// one line
{ key: 'i1|v2|a1,a5', itemId: 'i1', variantId: 'v2', addonIds: ['a1', 'a5'], qty: 2 }

// customise sheet draft
const [sel, setSel] = useState(() => defaultSelection(item)); // { variantId: 'v1', addonIds: ['a1']
}
```

Senior note: derive, don't store.

- **Not stored:** line price, item total, taxes, delivery, grand total, cart count, qty-per-item for the card, the validation errors, the live price in the sheet. All are computed from `lines` + `menu`. Storing them means keeping two copies in sync, and they will drift.
- Lines store **ids, not prices**. Price comes from the menu (single source of truth). If the menu updates a price, the cart shows the new one. (Trade-off in the wrap-up.)
- **Ownership:** cart state is global (card, cart panel, header badge all need it) → context. The draft selection is local to the sheet → `useState` there; it never touches the cart until "Add item".

5. Design the data structures and contracts

Example data (normalized, stable ids)

```
{
  restaurant: { id: 'r1', name: 'Pizza Palace', deliveryFee: 40, freeDeliveryAbove: 499 },
```

```
  categories: [
    { id: 'c1', name: 'Recommended', itemIds: ['i1', 'i2', 'i3'] },
    { id: 'c2', name: 'Pizzas',      itemIds: ['i1', 'i4'] },     // i1 appears twice
  ],
  items: {
    i1: {
      id: 'i1', name: 'Margherita', price: 199, veg: true, available: true,
      variants: [{ id: 'v1', name: 'Regular', price: 199 }, { id: 'v2', name: 'Medium', price: 349
}],
      addonGroups: [
        { id: 'g1', name: 'Crust', min: 1, max: 1, options: [{ id: 'a1', name: 'Hand tossed', price:
0 }, ...] },
        { id: 'g2', name: 'Extra toppings', min: 0, max: 3, options: [{ id: 'a3', name: 'Olives',
price: 35 }, ...] },
      ],
    },
  },
}
```

Why this shape:

- `items` **by id:** "Margherita" is in *Recommended* and *Pizzas*. With one item object, both cards show the same qty, and cart lookups are O(1).
- **Stable ids** for items, variants, options: React keys and cart line keys must not depend on names or array index.
- `min` **/** `max` **on the group:** one generic rule covers radio (`1/1`), optional single (`0/1`), multi (`0/N`), required multi (`1/N`). New groups need no code.
- **Variant vs add-on are different:** variant price *replaces* the base, add-on price *adds*. Keeping them separate makes pricing obvious.

Contracts

Function	Input	Output
`fetchMenu()`	–	`Promise<Menu>` (above)
`lineKey(itemId, variantId, addonIds)`	ids	`'i1|v2|a1,a5'`
`unitPrice(item, variantId, addonIds)`	item + selection	number (₹)
`toggleAddon(group, addonIds, optionId)`	group rules + current ids	next ids (respects max)
`selectionErrors(item, addonIds)`	item + selection	`{ [groupId]: message }`
`getBill(lines, items, restaurant)`	cart + menu	`{ count, itemTotal, delivery, taxes, total }`

Function	Input	Output
`dispatch({ type: 'ADD', restaurantId, itemId, variantId, addonIds })`	action	new cart state

A real API would also need `POST /cart/validate` at checkout returning the server-computed bill (prices, availability, taxes). The client bill is for display only.

---

6. Trace the data flow

Key interaction: add a *Medium Margherita, Cheese burst, Paneer* when one identical line already exists.

```
User clicks "+" on the Margherita card
   │
   ▼
MenuItemCard: isCustomizable(item) → true → onRepeat(item)
   │
   ▼
Menu: setRepeatFor(item) → RepeatPrompt shows "Medium, Cheese burst, Paneer"
   │  user clicks "I'll choose"
   ▼
Menu: setCustomizing(item), setRepeatFor(null) → CustomizeModal opens
   │  sel = defaultSelection(item) = { variantId: 'v1', addonIds: ['a1'] }
   ▼
User picks Medium, Cheese burst, Paneer
   │  setSel → toggleAddon(...) → { variantId: 'v2', addonIds: ['a2', 'a5'] }
   │  render: unitPrice = 349 + 99 + 60 = ₹508, selectionErrors = {} → button enabled
   ▼
Submit → add('i1', sel) → dispatch ADD
   │
   ▼
cartReducer: key = lineKey('i1', 'v2', ['a2','a5']) = 'i1|v2|a2,a5'
   │  line with that key exists → qty 1 → 2 (capped at 10)
   ▼
New cart state → useEffect writes localStorage
   │  context value recomputed: qtyByItem.i1 = 2, bill = getBill(...)
   ▼
Output: card stepper shows 2, cart line "Medium, Cheese burst, Paneer × 2 = ₹1,016",
        item total ₹1,016 → delivery FREE, taxes ₹51, to pay ₹1,067
```

---

7. Important technical challenges

Challenge 1: Cart line identity with variants and add-ons

Problem: User adds "Margherita, Medium, Olives + Paneer", then later adds "Margherita, Medium, Paneer + Olives" (clicked in the other order). If lines are keyed by `itemId`, the second overwrites the first config. If every add creates a new line, the cart shows two identical rows.

Solution: Key each line by `itemId + variantId + sorted addonIds`. Sorting makes click order irrelevant. Same key → increment, new key → new line.

```
export const lineKey = (itemId, variantId, addonIds) =>
  [itemId, variantId ?? '-', [...addonIds].sort().join(',')].join('|');

// reducer ADD
const key = lineKey(itemId, variantId, addonIds);
const exists = base.lines.some((l) => l.key === key);
const lines = exists
  ? base.lines.map((l) => (l.key === key ? { ...l, qty: Math.min(l.qty + 1, MAX_QTY) } : l))
  : [...base.lines, { key, itemId, variantId, addonIds: [...addonIds].sort(), qty: 1 }];
```

Challenge 2: Add-on group rules (min / max) with a live price

Problem: "Crust: choose 1" must behave like a radio; "Toppings: up to 3" must stop at 3; a required group left empty must block "Add item". Writing per-group `if`s doesn't scale to a real menu with 10 groups.

Solution: One generic `toggleAddon` driven by the group's `max`, one `selectionErrors` driven by `min/max`, and price/errors derived on each render (never stored). The UI disables extra checkboxes when the max is hit, so the user sees the limit.

```
export function toggleAddon(group, addonIds, optionId) {
  const inGroup = new Set(group.options.map((o) => o.id));
  if (group.max === 1) return [...addonIds.filter((id) => !inGroup.has(id)), optionId]; // radio
  if (addonIds.includes(optionId)) return addonIds.filter((id) => id !== optionId);
  const picked = addonIds.filter((id) => inGroup.has(id)).length;
  return picked >= group.max ? addonIds : [...addonIds, optionId];
}

// in CustomizeModal render
const errors = selectionErrors(item, sel.addonIds);
const price = unitPrice(item, sel.variantId, sel.addonIds); // base or variant + add-ons
```

Challenge 3: The card stepper for an item with many cart lines

Problem: The Margherita card shows one stepper, but the cart has two Margherita lines (Medium and Large). What do **+** and **−** do? Which line changes?

Solution:

- Card qty = **sum** of all its lines (derived `qtyByItem`).
- **+** on a customisable item → "Repeat last customisation?" (increments the last line) or "I'll choose" (opens the sheet).
- **−** with one line → decrement it. With 2+ lines → we can't guess, so tell the user to remove it from the cart (Swiggy does the same).

```
const qtyByItem = {};
for (const l of cart.lines) qtyByItem[l.itemId] = (qtyByItem[l.itemId] ?? 0) + l.qty;

const handleInc = () => (custom ? onRepeat(item) : inc(itemLines[0].key));
const handleDec = () => {
  if (itemLines.length > 1) return alert('This item has multiple customisations. Remove the one you want from the cart.');
  dec(itemLines[0].key);
};
```

Challenge 4: Derived bill that stays correct

Problem: If you store `total` in state, every action (inc, dec, clear, price change, item going out of stock) must remember to update it. One missed path → wrong amount to pay.

Solution: Store only lines; compute the bill from lines + menu. Skip lines whose item is missing or sold out (e.g. a stale cart restored from localStorage).

```
export function getBill(lines, items, restaurant) {
  const valid = lines.filter((l) => items[l.itemId]?.available);
  const itemTotal = valid.reduce((s, l) => s + unitPrice(items[l.itemId], l.variantId, l.addonIds) * l.qty, 0);
  const delivery = itemTotal === 0 || itemTotal >= restaurant.freeDeliveryAbove ? 0 : restaurant.deliveryFee;
  const taxes = Math.round(itemTotal * 0.05);
  return { count: valid.reduce((s, l) => s + l.qty, 0), itemTotal, delivery, taxes, total: itemTotal + delivery + taxes };
}
```

---

8. Implement in the correct order

#	Step	Working result
1	Mock data + `fetchMenu`, `App` with loading / error / retry	Menu JSON loads
2	`Menu` renders categories and item cards (name, price, veg mark)	Static menu visible
3	`cartReducer` (ADD / INC / DEC / CLEAR) + `CartProvider` + `useCart`	Cart state exists
4	ADD button + `QtyStepper` for **simple** items	Add / remove simple items
5	`Cart` panel with lines + `getBill`	Live bill
6	`utils/cart.js`: `lineKey`, `unitPrice`, `defaultSelection`	Variant-aware keys and price
7	`Modal` + `CustomizeModal`: variants (radios) + live price	Add sizes
8	Add-on groups: `toggleAddon`, `selectionErrors`, disabled states	Toppings with rules
9	`RepeatPrompt` + card **+ / −** rules for multi-line items	Swiggy-like stepper
10	Search + Veg only filter, empty-results message	Filtering
11	localStorage persistence (lazy init + effect, try/catch)	Survives refresh
12	Edge cases: sold out, max qty, free-delivery hint, restaurant switch	Polished

9. Cover all states and edge cases

State / edge case	Handling
Initial / loading	"Loading menu…"
Error	"Couldn't load the menu" + Retry button
Success	Menu + cart side by side (stacked on mobile)
Empty cart	"Your cart is empty" card
Empty search / filter	"No dishes match your filters", empty categories hidden
Sold-out item	"Sold out" label, no ADD
Persisted line for sold-out / removed item	Greyed "(unavailable)" or hidden, excluded from bill and count
Same config added twice (any click order)	Merged into one line via sorted key
Required group empty	Inline `role="alert"` error, submit disabled
Max add-ons reached	Remaining checkboxes disabled
Max qty (10) per line	`+` disabled in cart, reducer caps anyway
Qty reaches 0	Line removed; last line removed → cart reset (restaurantId cleared)

State / edge case	Handling
`−` on card with 2+ customisations	Message to remove from cart
Repeat prompt but line removed meanwhile	Prompt renders nothing (`if (!last) return null`)
Cart from another restaurant	Confirm "Replace?"; reducer starts a fresh cart
localStorage corrupted / blocked / full	try/catch → fallback to empty cart / in-memory only
Item total below threshold	₹40 delivery + "Add ₹X more for free delivery"
Item in two categories	Same object, same qty everywhere
Escape / backdrop click	Closes sheet, draft discarded

---

10. Final structure

```
_74_food_menu_cart/
├── index.html
├── package.json
└── src/
    ├── main.jsx
    ├── index.css                # @import "tailwindcss";
    ├── App.jsx                  # fetch + loading/error + layout
    ├── data/
    │   └── menu.js              # normalized mock menu + fetchMenu()
    ├── utils/
    │   └── cart.js              # pure: lineKey, unitPrice, toggleAddon, selectionErrors, getBill
    ├── context/
    │   ├── cartReducer.js       # ADD / INC / DEC / CLEAR
    │   └── CartContext.jsx      # provider, persistence, derived values, useCart
    └── components/
        ├── Menu.jsx             # filters, sections, which sheet is open
        ├── MenuItemCard.jsx     # ADD / stepper / customise / repeat
        ├── CustomizeModal.jsx   # variants + add-on groups + live price
        ├── RepeatPrompt.jsx     # repeat last vs choose new
        ├── Cart.jsx             # lines + bill
        ├── QtyStepper.jsx       # generic − qty +
        └── Modal.jsx            # generic dialog (Escape, backdrop)
```

---

11. Full working code

`src/data/menu.js`

```
// Mock API: normalized menu. Items live once in `items`; categories only hold ids,
// so an item listed in two categories shares one cart count.
export const MENU = {
  restaurant: { id: 'r1', name: 'Pizza Palace', deliveryFee: 40, freeDeliveryAbove: 499 },
  categories: [
    { id: 'c1', name: 'Recommended', itemIds: ['i1', 'i2', 'i3'] },
    { id: 'c2', name: 'Pizzas', itemIds: ['i1', 'i4'] },
    { id: 'c3', name: 'Beverages', itemIds: ['i5', 'i6'] },
  ],
  items: {
    i1: {
      id: 'i1', name: 'Margherita', desc: 'Classic cheese and tomato', price: 199, veg: true,
available: true,
      // variant price REPLACES the base price
      variants: [
        { id: 'v1', name: 'Regular', price: 199 },
        { id: 'v2', name: 'Medium', price: 349 },
        { id: 'v3', name: 'Large', price: 499 },
      ],
      // add-on price is ADDED on top; min/max are the group's selection rules
      addonGroups: [
        {
          id: 'g1', name: 'Crust', min: 1, max: 1,
          options: [
            { id: 'a1', name: 'Hand tossed', price: 0 },
            { id: 'a2', name: 'Cheese burst', price: 99 },
          ],
        },
        {
          id: 'g2', name: 'Extra toppings', min: 0, max: 3,
          options: [
            { id: 'a3', name: 'Olives', price: 35 },
            { id: 'a4', name: 'Jalapeno', price: 35 },
            { id: 'a5', name: 'Paneer', price: 60 },
            { id: 'a6', name: 'Extra cheese', price: 50 },
          ],
        },
      ],
    },
    i2: { id: 'i2', name: 'Garlic Bread', desc: 'Toasted with herb butter', price: 129, veg: true,
```

```
available: true },
    i3: {
      id: 'i3', name: 'Chicken Wings', desc: '6 pcs, smoky BBQ', price: 249, veg: false, available:
true,
      addonGroups: [
        {
          id: 'g3', name: 'Dips', min: 0, max: 2,
          options: [
            { id: 'a7', name: 'Mayo', price: 20 },
            { id: 'a8', name: 'Peri peri', price: 25 },
            { id: 'a9', name: 'Cheese', price: 30 },
          ],
        },
      ],
    },
    i4: {
      id: 'i4', name: 'Farmhouse', desc: 'Onion, capsicum, mushroom', price: 299, veg: true,
available: false,
      variants: [
        { id: 'v4', name: 'Regular', price: 299 },
        { id: 'v5', name: 'Large', price: 599 },
      ],
    },
    i5: { id: 'i5', name: 'Coke (500 ml)', desc: 'Chilled', price: 60, veg: true, available: true },
    i6: { id: 'i6', name: 'Cold Coffee', desc: 'With ice cream', price: 120, veg: true, available:
true },
  },
};

export function fetchMenu() {
  return new Promise((resolve) => setTimeout(() => resolve(structuredClone(MENU)), 500));
}
```

src/utils/cart.js

```
export const MAX_QTY = 10;
const TAX_RATE = 0.05;

// Same item + same variant + same add-ons (in any click order) => same cart line.
export const lineKey = (itemId, variantId, addonIds) =>
  [itemId, variantId ?? '-', [...addonIds].sort().join(',')].join('|');

export const isCustomizable = (item) => Boolean(item.variants?.length || item.addonGroups?.length);
```

```
const allAddons = (item) => (item.addonGroups ?? []).flatMap((g) => g.options);

export function unitPrice(item, variantId, addonIds) {
  const base = item.variants?.find((v) => v.id === variantId)?.price ?? item.price;
  const addons = allAddons(item)
    .filter((o) => addonIds.includes(o.id))
    .reduce((sum, o) => sum + o.price, 0);
  return base + addons;
}

export function defaultSelection(item) {
  return {
    variantId: item.variants?.[0]?.id ?? null,
    // pre-pick the first option of every "choose exactly 1" group
    addonIds: (item.addonGroups ?? [])
      .filter((g) => g.min === 1 && g.max === 1)
      .map((g) => g.options[0].id),
  };
}

export function toggleAddon(group, addonIds, optionId) {
  const inGroup = new Set(group.options.map((o) => o.id));
  if (group.max === 1) return [...addonIds.filter((id) => !inGroup.has(id)), optionId]; // radio
  if (addonIds.includes(optionId)) return addonIds.filter((id) => id !== optionId);
  const picked = addonIds.filter((id) => inGroup.has(id)).length;
  return picked >= group.max ? addonIds : [...addonIds, optionId];
}

export function selectionErrors(item, addonIds) {
  const errors = {};
  for (const g of item.addonGroups ?? []) {
    const picked = g.options.filter((o) => addonIds.includes(o.id)).length;
    if (picked < g.min) errors[g.id] = `Pick at least ${g.min}`;
    if (picked > g.max) errors[g.id] = `Pick at most ${g.max}`;
  }
  return errors;
}

export function describeLine(item, line) {
  const variant = item.variants?.find((v) => v.id === line.variantId)?.name;
  const addons = allAddons(item).filter((o) => line.addonIds.includes(o.id)).map((o) => o.name);
  return [variant, ...addons].filter(Boolean).join(', ');
}

// Everything money-related is derived from cart lines + the menu (single source of truth).
```

```
export function getBill(lines, items, restaurant) {
  const valid = lines.filter((l) => items[l.itemId]?.available);
  const itemTotal = valid.reduce(
    (sum, l) => sum + unitPrice(items[l.itemId], l.variantId, l.addonIds) * l.qty,
    0,
  );
  const delivery = itemTotal === 0 || itemTotal >= restaurant.freeDeliveryAbove ? 0 :
restaurant.deliveryFee;
  const taxes = Math.round(itemTotal * TAX_RATE);
  return {
    count: valid.reduce((sum, l) => sum + l.qty, 0),
    itemTotal,
    delivery,
    taxes,
    total: itemTotal + delivery + taxes,
  };
}

export const formatINR = (n) => `₹${n.toLocaleString('en-IN')}`;
```

src/context/cartReducer.js

```
import { lineKey, MAX_QTY } from '../utils/cart.js';

export const initialCart = { restaurantId: null, lines: [] };

export function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD': {
      const { restaurantId, itemId, variantId = null, addonIds = [] } = action;
      // a cart belongs to one restaurant: adding from another one starts fresh
      const base = state.restaurantId && state.restaurantId !== restaurantId ? initialCart : state;
      const key = lineKey(itemId, variantId, addonIds);
      const exists = base.lines.some((l) => l.key === key);
      const lines = exists
        ? base.lines.map((l) => (l.key === key ? { ...l, qty: Math.min(l.qty + 1, MAX_QTY) } : l))
        : [...base.lines, { key, itemId, variantId, addonIds: [...addonIds].sort(), qty: 1 }];
      return { restaurantId, lines };
    }
    case 'INC':
      return {
        ...state,
        lines: state.lines.map((l) =>
          l.key === action.key ? { ...l, qty: Math.min(l.qty + 1, MAX_QTY) } : l,
```

```
        ),
      };
    case 'DEC': {
      const lines = state.lines
        .map((l) => (l.key === action.key ? { ...l, qty: l.qty - 1 } : l))
        .filter((l) => l.qty > 0);
      return lines.length ? { ...state, lines } : initialCart;
    }
    case 'CLEAR':
      return initialCart;
    default:
      return state;
  }
}
```

src/context/CartContext.jsx

```
import { createContext, useContext, useEffect, useMemo, useReducer } from 'react';
import { cartReducer, initialCart } from './cartReducer.js';
import { getBill } from '../utils/cart.js';

const CartContext = createContext(null);
const STORAGE_KEY = 'food-cart';

function loadCart() {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY)) ?? initialCart;
  } catch {
    return initialCart; // corrupted JSON or storage blocked
  }
}

export function CartProvider({ menu, children }) {
  const [cart, dispatch] = useReducer(cartReducer, undefined, loadCart);

  useEffect(() => {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(cart));
    } catch {
      /* quota exceeded / private mode: cart still works in memory */
    }
  }, [cart]);

  const value = useMemo(() => {
```

```
    const qtyByItem = {};
    for (const l of cart.lines) qtyByItem[l.itemId] = (qtyByItem[l.itemId] ?? 0) + l.qty;

    const add = (itemId, { variantId = null, addonIds = [] } = {}) => {
      const otherRestaurant = cart.restaurantId && cart.restaurantId !== menu.restaurant.id;
      if (otherRestaurant && !window.confirm('Your cart has items from another restaurant. Replace
them?')) return;
      dispatch({ type: 'ADD', restaurantId: menu.restaurant.id, itemId, variantId, addonIds });
    };

    return {
      cart,
      qtyByItem,
      bill: getBill(cart.lines, menu.items, menu.restaurant),
      lastLine: (itemId) => cart.lines.findLast((l) => l.itemId === itemId),
      add,
      inc: (key) => dispatch({ type: 'INC', key }),
      dec: (key) => dispatch({ type: 'DEC', key }),
      clear: () => dispatch({ type: 'CLEAR' }),
    };
  }, [cart, menu]);

  return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart must be used inside <CartProvider>');
  return ctx;
}
```

`src/components/Modal.jsx`

```
import { useEffect } from 'react';

export default function Modal({ title, onClose, children }) {
  useEffect(() => {
    const onKey = (e) => e.key === 'Escape' && onClose();
    window.addEventListener('keydown', onKey);
    return () => window.removeEventListener('keydown', onKey);
  }, [onClose]);

  return (
    <div className="fixed inset-0 z-10 flex items-end justify-center bg-black/50 sm:items-center"
```

```
onClick={onClose}>
      <div
        role="dialog"
        aria-modal="true"
        aria-label={title}
        onClick={(e) => e.stopPropagation()}
        className="max-h-[90vh] w-full max-w-md overflow-y-auto rounded-t-2xl bg-white p-5
sm:rounded-2xl"
      >
        <div className="mb-3 flex items-center justify-between">
          <h2 className="text-lg font-semibold">{title}</h2>
          <button autoFocus aria-label="Close" onClick={onClose} className="px-2 text-xl">
            ×
          </button>
        </div>
        {children}
      </div>
    </div>
  );
}
```

`src/components/QtyStepper.jsx`

```
export default function QtyStepper({ qty, label, onInc, onDec, max = Infinity }) {
  return (
    <div className="inline-flex items-center rounded-lg border border-green-600 font-semibold text-
green-700">
      <button aria-label={`Decrease ${label}`} onClick={onDec} className="px-3 py-1">
        –
      </button>
      <span aria-live="polite" className="w-6 text-center">
        {qty}
      </span>
      <button
        aria-label={`Increase ${label}`}
        onClick={onInc}
        disabled={qty >= max}
        className="px-3 py-1 disabled:opacity-40"
      >
        +
      </button>
    </div>
  );
}
```

`src/components/MenuItemCard.jsx`

```
import { useCart } from '../context/CartContext.jsx';
import { formatINR, isCustomizable } from '../utils/cart.js';
import QtyStepper from './QtyStepper.jsx';

const VegMark = ({ veg }) => (
  <span
    aria-label={veg ? 'Veg' : 'Non-veg'}
    className={`inline-block h-3 w-3 rounded-sm border-2 ${veg ? 'border-green-600 bg-green-600/30'
: 'border-red-600 bg-red-600/30'}`}
  />
);

export default function MenuItemCard({ item, onCustomize, onRepeat }) {
  const { cart, qtyByItem, add, inc, dec } = useCart();
  const qty = qtyByItem[item.id] ?? 0;
  const custom = isCustomizable(item);
  const itemLines = cart.lines.filter((l) => l.itemId === item.id);

  const handleAdd = () => (custom ? onCustomize(item) : add(item.id));
  const handleInc = () => (custom ? onRepeat(item) : inc(itemLines[0].key));
  const handleDec = () => {
    // the card can't know WHICH customization to remove
    if (itemLines.length > 1) return alert('This item has multiple customisations. Remove the one
you want from the cart.');
    dec(itemLines[0].key);
  };

  return (
    <li className="flex justify-between gap-4 border-b py-4">
      <div>
        <VegMark veg={item.veg} />
        <h3 className="font-medium">{item.name}</h3>
        <p className="text-sm">{formatINR(item.variants?.[0]?.price ?? item.price)}</p>
        <p className="text-sm text-gray-500">{item.desc}</p>
      </div>
      <div className="flex shrink-0 flex-col items-center gap-1">
        {!item.available ? (
          <span className="text-sm font-medium text-gray-400">Sold out</span>
        ) : qty === 0 ? (
          <button
            onClick={handleAdd}
            aria-label={`Add ${item.name}`}
            className="rounded-lg border border-gray-300 px-6 py-1 font-semibold text-green-700
```

```
shadow-sm"
          >
            ADD
          </button>
        ) : (
          <QtyStepper qty={qty} label={item.name} onInc={handleInc} onDec={handleDec} />
        )}
        {custom && item.available && <span className="text-xs text-gray-500">Customisable</span>}
      </div>
    </li>
  );
}
```

src/components/CustomizeModal.jsx

```
import { useState } from 'react';
import { useCart } from '../context/CartContext.jsx';
import { defaultSelection, formatINR, selectionErrors, toggleAddon, unitPrice } from
'../utils/cart.js';
import Modal from './Modal.jsx';

export default function CustomizeModal({ item, onClose }) {
  const { add } = useCart();
  const [sel, setSel] = useState(() => defaultSelection(item));

  // derived on every render: never stored, never stale
  const errors = selectionErrors(item, sel.addonIds);
  const valid = Object.keys(errors).length === 0;
  const price = unitPrice(item, sel.variantId, sel.addonIds);

  const submit = (e) => {
    e.preventDefault();
    if (!valid) return;
    add(item.id, sel);
    onClose();
  };

  return (
    <Modal title={`Customise ${item.name}`} onClose={onClose}>
      <form onSubmit={submit} className="space-y-4">
        {item.variants?.length > 0 && (
          <fieldset>
            <legend className="font-medium">Size</legend>
            {item.variants.map((v) => (
```

```
              <label key={v.id} className="flex justify-between py-1">
                <span>
                  <input
                    type="radio"
                    name="variant"
                    checked={sel.variantId === v.id}
                    onChange={() => setSel((s) => ({ ...s, variantId: v.id }))}
                  />{' '}
                  {v.name}
                </span>
                <span>{formatINR(v.price)}</span>
              </label>
            ))}
          </fieldset>
        )}

        {item.addonGroups?.map((g) => {
          const picked = g.options.filter((o) => sel.addonIds.includes(o.id)).length;
          return (
            <fieldset key={g.id}>
              <legend className="font-medium">
                {g.name}{' '}
                <span className="text-xs text-gray-500">{g.max === 1 ? '(choose 1)' : `(up to
${g.max})`}</span>
              </legend>
              {g.options.map((o) => {
                const checked = sel.addonIds.includes(o.id);
                return (
                  <label key={o.id} className="flex justify-between py-1">
                    <span>
                      <input
                        type={g.max === 1 ? 'radio' : 'checkbox'}
                        name={g.id}
                        checked={checked}
                        disabled={!checked && g.max > 1 && picked >= g.max}
                        onChange={() => setSel((s) => ({ ...s, addonIds: toggleAddon(g, s.addonIds,
o.id) }))}
                      />{' '}
                      {o.name}
                    </span>
                    <span>+{formatINR(o.price)}</span>
                  </label>
                );
              })}
              {errors[g.id] && (
```

```
                <p role="alert" className="text-xs text-red-600">
                  {errors[g.id]}
                </p>
              )}
            </fieldset>
          );
        })}

        <button
          type="submit"
          disabled={!valid}
          className="w-full rounded-lg bg-green-600 py-2 font-semibold text-white disabled:opacity-
50"
        >
          Add item · {formatINR(price)}
        </button>
      </form>
    </Modal>
  );
}
```

`src/components/RepeatPrompt.jsx`

```
import { useCart } from '../context/CartContext.jsx';
import { describeLine } from '../utils/cart.js';
import Modal from './Modal.jsx';

export default function RepeatPrompt({ item, onChooseNew, onClose }) {
  const { lastLine, inc } = useCart();
  const last = lastLine(item.id);
  if (!last) return null; // line removed from the cart while the prompt was open

  const repeat = () => {
    inc(last.key);
    onClose();
  };

  return (
    <Modal title="Repeat last customisation?" onClose={onClose}>
      <p className="mb-4 text-sm text-gray-600">
        {item.name} · {describeLine(item, last) || 'No add-ons'}
      </p>
      <div className="flex gap-3">
        <button onClick={onChooseNew} className="flex-1 rounded-lg border border-green-600 py-2
```

```
font-semibold text-green-700">
          I&apos;ll choose
        </button>
        <button onClick={repeat} className="flex-1 rounded-lg bg-green-600 py-2 font-semibold text-
white">
          Repeat
        </button>
      </div>
    </Modal>
  );
}
```

`src/components/Menu.jsx`

```
import { useState } from 'react';
import MenuItemCard from './MenuItemCard.jsx';
import CustomizeModal from './CustomizeModal.jsx';
import RepeatPrompt from './RepeatPrompt.jsx';

export default function Menu({ menu }) {
  const [query, setQuery] = useState('');
  const [vegOnly, setVegOnly] = useState(false);
  const [customizing, setCustomizing] = useState(null); // item in the customise sheet
  const [repeatFor, setRepeatFor] = useState(null); // item in the "repeat?" prompt

  const q = query.trim().toLowerCase();
  const sections = menu.categories
    .map((c) => ({
      ...c,
      items: c.itemIds
        .map((id) => menu.items[id])
        .filter((i) => (!vegOnly || i.veg) && i.name.toLowerCase().includes(q)),
    }))
    .filter((c) => c.items.length > 0);

  return (
    <section>
      <h1 className="mb-4 text-2xl font-bold">{menu.restaurant.name}</h1>
      <div className="mb-4 flex flex-wrap items-center gap-4">
        <input
          type="search"
          aria-label="Search menu"
          placeholder="Search for dishes"
          value={query}
```

```
          onChange={(e) => setQuery(e.target.value)}
          className="flex-1 rounded-lg border px-3 py-2"
        />
        <label className="flex items-center gap-2 text-sm">
          <input type="checkbox" checked={vegOnly} onChange={(e) => setVegOnly(e.target.checked)} />
Veg only
        </label>
      </div>

      {sections.length === 0 ? (
        <p className="py-10 text-center text-gray-500">No dishes match your filters.</p>
      ) : (
        sections.map((c) => (
          <div key={c.id} className="mb-6">
            <h2 className="text-lg font-semibold">
              {c.name} ({c.items.length})
            </h2>
            <ul>
              {c.items.map((item) => (
                <MenuItemCard key={item.id} item={item} onCustomize={setCustomizing} onRepeat=
{setRepeatFor} />
              ))}
            </ul>
          </div>
        ))
      )}

      {customizing && <CustomizeModal item={customizing} onClose={() => setCustomizing(null)} />}
      {repeatFor && (
        <RepeatPrompt
          item={repeatFor}
          onChooseNew={() => {
            setCustomizing(repeatFor);
            setRepeatFor(null);
          }}
          onClose={() => setRepeatFor(null)}
        />
      )}
    </section>
  );
}
```

`src/components/Cart.jsx`

```
import { useCart } from '../context/CartContext.jsx';
import { describeLine, formatINR, MAX_QTY, unitPrice } from '../utils/cart.js';
import QtyStepper from './QtyStepper.jsx';

export default function Cart({ menu }) {
  const { cart, bill, inc, dec, clear } = useCart();
  const { freeDeliveryAbove } = menu.restaurant;

  if (cart.lines.length === 0) {
    return (
      <aside className="h-fit rounded-xl border p-5 text-center">
        <h2 className="font-semibold">Your cart is empty</h2>
        <p className="text-sm text-gray-500">Add dishes from the menu to see them here.</p>
      </aside>
    );
  }

  return (
    <aside aria-label="Cart" className="h-fit rounded-xl border p-5 md:sticky md:top-4">
      <div className="mb-3 flex items-center justify-between">
        <h2 className="font-semibold">Cart · {bill.count} items</h2>
        <button onClick={clear} className="text-sm text-red-600">
          Clear
        </button>
      </div>

      <ul className="space-y-3">
        {cart.lines.map((line) => {
          const item = menu.items[line.itemId];
          if (!item) return null; // persisted line for an item no longer on the menu
          return (
            <li key={line.key} className={`flex items-center gap-2 ${item.available ? '' : 'opacity-
50'}`}>
              <div className="flex-1">
                <p className="text-sm font-medium">
                  {item.name} {!item.available && <span className="text-red-600">(unavailable)
</span>}
                </p>
                <p className="text-xs text-gray-500">{describeLine(item, line)}</p>
              </div>
              <QtyStepper qty={line.qty} label={item.name} max={MAX_QTY} onInc={() => inc(line.key)}
onDec={() => dec(line.key)} />
              <span className="w-16 text-right text-sm">
```

```
                {formatINR(unitPrice(item, line.variantId, line.addonIds) * line.qty)}
              </span>
            </li>
          );
        })}
      </ul>

      <dl className="mt-4 space-y-1 border-t pt-3 text-sm">
        <div className="flex justify-between"><dt>Item total</dt><dd>{formatINR(bill.itemTotal)}
</dd></div>
        <div className="flex justify-between"><dt>Delivery fee</dt><dd>{bill.delivery ?
formatINR(bill.delivery) : 'FREE'}</dd></div>
        <div className="flex justify-between"><dt>Taxes (5%)</dt><dd>{formatINR(bill.taxes)}</dd>
</div>
        <div className="flex justify-between border-t pt-2 font-semibold"><dt>To pay</dt><dd>
{formatINR(bill.total)}</dd></div>
      </dl>

      {bill.itemTotal > 0 && bill.itemTotal < freeDeliveryAbove && (
        <p className="mt-2 text-xs text-green-700">
          Add {formatINR(freeDeliveryAbove - bill.itemTotal)} more for free delivery
        </p>
      )}

      <button
        disabled={bill.count === 0}
        className="mt-4 w-full rounded-lg bg-green-600 py-2 font-semibold text-white
disabled:opacity-50"
      >
        Checkout
      </button>
    </aside>
  );
}
```

src/App.jsx

```
import { useCallback, useEffect, useState } from 'react';
import { fetchMenu } from './data/menu.js';
import { CartProvider } from './context/CartContext.jsx';
import Menu from './components/Menu.jsx';
import Cart from './components/Cart.jsx';

export default function App() {
```

```
  const [status, setStatus] = useState('loading'); // loading | success | error
  const [menu, setMenu] = useState(null);

  const load = useCallback(() => {
    setStatus('loading');
    fetchMenu()
      .then((m) => {
        setMenu(m);
        setStatus('success');
      })
      .catch(() => setStatus('error'));
  }, []);

  useEffect(() => {
    load();
  }, [load]);

  if (status === 'loading') return <p className="p-10 text-center text-gray-500">Loading menu…</p>;
  if (status === 'error') {
    return (
      <div className="p-10 text-center">
        <p className="mb-3">Couldn&apos;t load the menu.</p>
        <button onClick={load} className="rounded-lg border px-4 py-2">
          Retry
        </button>
      </div>
    );
  }

  return (
    <CartProvider menu={menu}>
      <main className="mx-auto grid max-w-5xl gap-6 p-4 md:grid-cols-[1fr_340px]">
        <Menu menu={menu} />
        <Cart menu={menu} />
      </main>
    </CartProvider>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
```

```
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

`src/index.css`

```
@import "tailwindcss";
```

---

12. Wrap-up

Edge cases handled

- Same customisation in any click order merges into one line; different ones stay separate.
- Add-on min/max enforced in the UI (disabled options, inline errors) and in the logic.
- Card stepper for multi-line items (repeat prompt, safe decrement).
- Sold-out and stale persisted items excluded from the bill.
- Max qty, free-delivery threshold, empty cart, empty search, fetch error with retry, broken localStorage.
- Single-restaurant cart with replace confirmation.

Trade-offs to mention

- **Price from menu vs price snapshot in the line:** deriving from the menu keeps one source of truth and shows price changes immediately; storing a snapshot shows what the user saw when adding. Real apps snapshot *and* re-validate on the server at checkout.
- **Context vs Redux / Zustand:** one context is enough for a page. Every consumer re-renders on any cart change; fine for ~100 cards. At scale: split context or use a store with selectors.
- **Array of lines vs map by key:** array keeps insertion order and is tiny; lookups are O(lines), which is ≤ 20 in practice.
- **Rupees as integers:** fine for whole-rupee menus. With paise or discounts, store paise to avoid floating-point errors.
- **`alert` / `confirm`:** fast for the interview; a real app uses a toast and a custom dialog.
- **Client-side bill is display-only:** never trust it for payment.

What to add with more time

- **Tests:** `node:test` / Vitest for `lineKey`, `toggleAddon`, `getBill`, reducer; RTL test for "add same config twice → one line".

- **Accessibility:** focus trap and focus return in `Modal`, body scroll lock.
- **UX:** sticky category nav with scrollspy, "View cart" floating bar on mobile, edit customisation from the cart, item images with lazy loading.
- **Data:** server cart sync + `/cart/validate`, coupons, packaging charges, nested add-ons (add-on that unlocks another group).
- **Performance:** `React.memo` on `MenuItemCard` with per-item qty prop, virtualized list for 1000+ items, debounced search.
- **Multi-tab:** `storage` event to sync the cart across tabs.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why does a variant replace the base price but an add-on adds to it?

A size is a different product (Medium pizza is ₹349, not ₹199 + something), so it replaces. An add-on is extra on top of whatever size you picked, so it adds. Keeping them as two separate fields (`variants`, `addonGroups`) makes `unitPrice` a two-line function.

Q2. What if an item has no variants but has add-ons, or neither?

`unitPrice` falls back to `item.price` when no variant is found (`?.price ?? item.price`). `isCustomizable` is false for items with neither, so ADD adds directly with `variantId: null, addonIds: []`.

Q3. Why not allow items from two restaurants in one cart?

One order = one restaurant = one delivery partner and one kitchen. Mixing breaks delivery fee, ETA and payment split. So the cart stores `restaurantId` and we confirm before replacing.

State design

Q4. Why don't you store `total` or the line price in state?

They are pure functions of `lines` + `menu`. Storing them creates a second copy that every action must keep in sync; one missed path shows a wrong amount. Deriving is O(lines), which is tiny.

Q5. Why is the customise selection local state and not in the cart context?

It is a draft. Until "Add item" it must not affect the cart, and it is thrown away on close. Local `useState` means no cleanup and no global re-renders while the user clicks toppings.

Q6. Why `useReducer` instead of several `useState`s?

Cart updates depend on the previous state (merge-or-append, cap qty, reset on empty) and the reducer is a pure function I can unit test without React. All transitions live in one place.

Q7. Why is the menu normalized (`items` by id + `itemIds` in categories)?

The same item can be in "Recommended" and "Pizzas". With one object, both cards show the same qty and availability updates in one place. Cart lookups `items[line.itemId]` are O(1).

Challenge 1: line identity

Q8. Why sort the add-on ids in the key?

Selection order depends on click order. `['a5','a3']` and `['a3','a5']` are the same pizza. Sorting gives one canonical key so they merge.

Q9. What if add-ons could be picked more than once (2× extra cheese)?

Change `addonIds: string[]` to `addons: { [id]: count }` and build the key from sorted `id:count` pairs, e.g. `a6:2,a3:1`. Price multiplies by count. `toggleAddon` becomes increment / decrement with a per-option max.

Q10. How would you let the user edit a customisation from the cart?

Open `CustomizeModal` with the line's `variantId/addonIds` as initial `sel`, then dispatch `REPLACE { oldKey, ...newSel }`: remove the old line and add the new config (merging if that key already exists, summing qty).

Challenge 2: add-on rules

Q11. What if a group is optional single choice (`min: 0, max: 1`)?

A radio can't be unchecked, so render it as a checkbox when `min === 0`, and in `toggleAddon` clicking the selected one removes it. Only `min: 1, max: 1` should be a radio.

Q12. Why validate in `selectionErrors` if the UI already disables options?

UI disabling only stops the *max* case. *Min* (required group) still needs a check, and the logic must be correct even if the UI changes or data comes in with defaults that violate rules. The submit handler checks `valid` again.

Challenge 3: card stepper

Q13. Why does "−" on the card not just remove the last line?

The user may want to remove the Medium, not the Large. Guessing removes the wrong item and they might not notice before paying. Sending them to the cart, where each line is visible, avoids silent mistakes.

Implementation steps

Q14. How do you persist the cart without a flash of empty cart?

Lazy initializer `useReducer(cartReducer, undefined, loadCart)` reads localStorage synchronously on the first render, so there is no empty → filled flicker. Writing happens in a `useEffect` on `[cart]`, wrapped in try/catch.

Q15. What if the stored cart has an item that was removed or is now sold out?

`getBill` filters to `items[l.itemId]?.available`, so it is not charged. The cart shows sold-out lines greyed with "(unavailable)" and hides lines whose item no longer exists. Checkout could also auto-prune them.

Q16. How would you sync the cart between two open tabs?

Listen to the `storage` event (fires in *other* tabs on `localStorage.setItem`) and dispatch a `HYDRATE` action with the parsed value.

Performance

Q17. What if the menu has 2,000 items?

Each cart change re-renders every card because they all read context. Fixes: pass `qty` as a prop and `React.memo` the card; or use Zustand / `useSyncExternalStore` with per-item selectors; plus list virtualization (`react-window`) and debounced search with a precomputed lowercase name.

Q18. What if the menu API is slow?

Show a skeleton of categories instead of a text loader, cache the last menu (stale-while-revalidate), and let users browse the cached menu while it revalidates. Prices are re-validated at checkout anyway.

Trade-offs / "why not X?"

Q19. Why not store the price in the cart line?

Then a price change on the server wouldn't show until checkout, and you have two prices to reconcile. I derive from the menu; in production I would *also* snapshot `priceAtAdd` to show "price changed" warnings and let the server compute the final bill.

Q20. Why not Redux?

For one page with one slice of global state, Context + `useReducer` gives the same reducer pattern without setup. I'd move to Redux Toolkit / Zustand when many pages share the cart, I need devtools/middleware, or re-render cost shows up.

Q21. How do you avoid floating-point errors in money?

Keep amounts as integers in the smallest unit (paise) and round only taxes once, at the end. Never add `0.1 + 0.2` style decimals. Format with `toLocaleString('en-IN')` only for display.

Q22. Is the client bill safe to send to payment?

No. The client can be tampered with. The server must recompute prices, availability, taxes, coupons and delivery from ids + qty and return the payable amount; the client only displays it.